

Wypożyczalnia pojazdów

Temat aplikacji

Projekt przedstawia prostą aplikację konsolową napisaną w języku Python. Aplikacja symuluje działanie wypożyczalni pojazdów. W programie można utworzyć pojazdy, klienta oraz wypożyczenie. Program pokazuje zastosowanie najważniejszych elementów programowania obiektowego: klas, obiektów, konstruktorów, właściwości, metod, identyfikatorów, enkapsulacji, dziedziczenia, polimorfizmu, abstrakcji oraz relacji między klasami.

Jak uruchomić projekt

W terminalu należy wpisać:

```
python wypożyczalnia_pojazdow.py
```

Lista klas

Klasa	Odpowiedzialność	Najważniejsze właściwości	Najważniejsze metody
<code>Rentable</code>	Klasa abstrakcyjna określająca kontrakt dla obiektów, które można wypożyczyć	<code>is_available</code>	<code>rent()</code> , <code>return_vehicle()</code>
<code>Vehicle</code>	Abstrakcyjna klasa bazowa dla pojazdów	<code>id</code> , <code>brand</code> , <code>model</code> , <code>hourly_rate</code> , <code>is_available</code>	<code>rent()</code> , <code>return_vehicle()</code> , <code>calculate_cost()</code> , <code>get_description()</code>
<code>Car</code>	Klasa szczegółowa reprezentująca samochód	<code>seats</code> , <code>has_air_conditioning</code>	<code>calculate_cost()</code> , <code>get_description()</code>
<code>Bike</code>	Klasa szczegółowa reprezentująca rower	<code>is_electric</code>	<code>calculate_cost()</code> , <code>get_description()</code>
<code>Customer</code>	Reprezentuje klienta wypożyczalni	<code>id</code> , <code>full_name</code> , <code>rentals</code>	<code>change_name()</code> , <code>add_rental()</code> , <code>__str__()</code>

Klasa	Odpowiedzialność	Najważniejsze właściwości	Najważniejsze metody
<code>Rental</code>	Reprezentuje pojedyncze wypożyczenie pojazdu	<code>id</code> , <code>customer</code> , <code>vehicle</code> , <code>hours</code> , <code>total_cost</code> , <code>is_finished</code>	<code>finish()</code> , <code>__str__()</code>
<code>RentalService</code>	Obsługuje logikę wypożyczalni	<code>vehicles</code> , <code>customers</code> , <code>rentals</code>	<code>add_vehicle()</code> , <code>add_customer()</code> , <code>create_rental()</code> , <code>finish_rental()</code>

Relacje między klasami

1. `Car` dziedziczy po klasie `Vehicle`.
2. `Bike` dziedziczy po klasie `Vehicle`.
3. `Vehicle` dziedziczy po klasie abstrakcyjnej `Rentable` i realizuje jej kontrakt.
4. `Rental` posiada obiekt klasy `Customer`, czyli wypożyczenie jest powiązane z konkretnym klientem.
5. `Rental` posiada obiekt klasy `Vehicle`, czyli wypożyczenie jest powiązane z konkretnym pojazdem.
6. `Customer` przechowuje kolekcję wypożyczeń w liście `_rentals`.
7. `RentalService` przechowuje kolekcje pojazdów, klientów i wypożyczeń.
8. Metoda `create_rental()` przyjmuje identyfikatory klienta i pojazdu, wyszukuje odpowiednie obiekty i tworzy między nimi relację wypożyczenia.

Cztery zasady OOP w projekcie

1. Enkapsulacja

Enkapsulacja została zastosowana przez ukrycie pól za pomocą podkreślenia, np. `_id`, `_brand`, `_model`, `_hourly_rate`, `_is_available`. Dostęp do danych odbywa się przez właściwości `@property`, np. `id`, `brand`, `model`, `is_available`.

Dostępność pojazdu nie jest zmieniana bezpośrednio. Można ją zmienić tylko przez metody `rent()` i `return_vehicle()`. Dzięki temu kod kontroluje poprawność operacji, np. nie pozwala wypożyczyć pojazdu, który jest już wypożyczony.

2. Dziedziczenie

Dziedziczenie występuje w relacji:

```
Car -> Vehicle
Bike -> Vehicle
Vehicle -> Rentable
```

Klasy `Car` i `Bike` przejmują wspólne cechy pojazdu z klasy `Vehicle`, np. identyfikator, markę, model, stawkę godzinową i dostępność.

3. Polimorfizm

Polimorfizm występuje przy metodach `calculate_cost()` oraz `get_description()`. W pliku `wypożyczalnia_pojazdow.py` program przechodzi po liście pojazdów i wywołuje te same metody dla różnych typów obiektów.

Przykład:

```
for vehicle in rental_service.vehicles:
    print(vehicle.get_description())
    print(vehicle.calculate_cost(3))
```

Dla samochodu koszt jest liczony inaczej niż dla roweru elektrycznego, mimo że metoda ma tę samą nazwę.

4. Abstrakcja

Abstrakcja została zastosowana przez klasę abstrakcyjną `Rentable` oraz abstrakcyjną klasę `Vehicle`.

Klasa `Rentable` określa, że obiekt możliwy do wypożyczenia musi posiadać:

- właściwość `is_available`,
- metodę `rent()`,
- metodę `return_vehicle()`.

Klasa `Vehicle` zawiera wspólne cechy pojazdów, ale nie reprezentuje konkretnego pojazdu. Konkretnie pojazdy są tworzone jako obiekty klas `Car` i `Bike`.

Lista kontrolna

- Projekt ma własny temat i opis: tak.
- Są klasy, obiekty, konstruktory, właściwości i metody: tak.
- Główne obiekty mają identyfikatory: tak, np. `CAR-001`, `BIKE-001`, `C-001`, `R-001`.
- Zastosowano enkapsulację: tak.
- Zastosowano dziedziczenie: tak.
- Zastosowano polimorfizm: tak.
- Zastosowano abstrakcję: tak.
- Są relacje między klasami, w tym kolekcja: tak.
- Kod można uruchomić: tak.
- Opis wskazuje, gdzie zastosowano elementy OOP: tak.

Przykładowe działanie programu

Program tworzy przykładowe pojazdy: samochód, rower standardowy i rower elektryczny. Następnie tworzy klienta, pokazuje listę pojazdów, oblicza koszt wypożyczenia każdego pojazdu na 3 godziny, tworzy wypożyczenie samochodu na 4 godziny, zmienia jego dostępność, a potem kończy wypożyczenie.